

O objetivo do projeto é analisar as emissões globais de CO₂, agrupando países por perfil de emissão com K-means, e visualizar os resultados em dashboards interativos.

Data Pipeline de Big Data

Infraestruturas Avançadas para
Ciência de Dados

Catarina Cruz (2020240694)
Íris Sousa (2021221971)

Índice

1. Introdução	2
2. Descrição do Problema e Dataset.....	2
3. Arquitetura e Tecnologias	3
3.1. Fluxo de dados	4
3.2. Containerização e orquestração	5
3.3. Tecnologias usadas.....	5
4. Análise e Avaliação de Resultados e Padrões de Clustering	6
4.1 Dashboard Interativo de Visualização.....	6
5. Testes.....	7
6. Conclusão.....	8
7. Bibliografia	8

1. Introdução

No âmbito da disciplina de Infraestruturas Avançadas para Ciência de Dados (IACD), foi-nos proposto o desenvolvimento de uma pipeline para processar um grande volume de informação em tempo real. Este projeto implementa essa pipeline, com a utilização de tecnologias de processamento distribuído para analisar dados de emissões de CO2.

O objetivo passa por processar dados, ao simularmos um fluxo contínuo, e aplicar algoritmos de clustering para identificar padrões entre os diferentes países. Os resultados são subsequentemente armazenados numa base de dados para análise e visualização. Este relatório descreve a arquitetura, a implementação e os resultados do sistema desenvolvido.

2. Descrição do Problema e Dataset

Começamos por escolher o dataset “CO2 and Greenhouse Gas Emissions” de *Our World in Data* [1], que nos fornece informações sobre as emissões de dióxido de carbono entre 1750 e 2024. Achamos que esta escolha seria bastante interessante devido à relevância atual das alterações climáticas e devido ao acesso do histórico temporal dos dados.

Originalmente, o dataset de aproximadamente 14 MB continha 79 variáveis, com informações sobre emissões per capita, por setor, consumo de energia, variáveis socioeconómicas, entre outras. Dada a complexidade e o número elevado de variáveis, optamos por realizar uma análise exploratória.

Descobrimos que:

- Os dados pré 1900 tinham mais de 60% de valores em falta;
- Na coluna country havia não só países individuais, como também agregados continentais e globais (ex: "World", "Asia"), o que poderia distorcer os resultados do clustering.

Para tratarmos destes problemas, filtramos o período temporal para 1900-2022 (pois não havia dados sobre o PIB depois de 2022 e essa era uma variável que queríamos manter) e removemos todas as entidades que não correspondiam a países individuais (identificadas pela ausência de iso_code). Das 79 variáveis originais, selecionamos 7 variáveis essenciais (país, ano, ISO code, emissões totais e per capita, PIB e população) por serem suficientes para caracterizar os perfis de emissão dos países, descartando variáveis relacionadas com setores específicos, fontes de energia e outras métricas secundárias. O resultado foi um dataset mais limpo e focado. Não tratamos dos valores em falta ainda existentes, pois esse tipo de processamento será feito a posteriori na fase de consumo Spark. Com estas variáveis, conseguimos realizar clustering que considera tanto as emissões, como o desenvolvimento económico e o contexto demográfico de cada país.

Tabela 1. Informações sobre o dataset.				
Versão do dataset	Linhas	Colunas	Período Temporal	Tamanho
Original	50 407	79	1750-2024	14MB
Processado	26 814	7	1900-2022	1.3MB

Tabela 2. Variáveis do dataset final.			
Variável	Tipo	Descrição	Justificação de escolha
<i>country</i>	String	Nome do país	Identificador principal
<i>year</i>	Inteiro	Ano (1900-2022)	Permite uma análise da evolução temporal
<i>iso_code</i>	String	Código ISO 3 letras	Facilita visualizações
<i>co2</i>	Float	Emissões totais de CO2 (milhões de toneladas)	Representam emissões em escalas diferentes (absoluta vs. normalizada)
<i>co2_per_capita</i>	Float	Emissões per capita	
<i>gdp</i>	Float	PIB (Produto Interno Bruto)	Permite correlacionar desenvolvimento económico com emissões
<i>population</i>	String	População do país	Essencial para compreender emissões per capita e escala dos países

3. Arquitetura e Tecnologias

Tal como já foi mencionado, seguimos uma pipeline de streaming de dados em tempo real baseada numa arquitetura onde cada componente tem uma responsabilidade específica, na Figura 1 podemos ver mais detalhes.

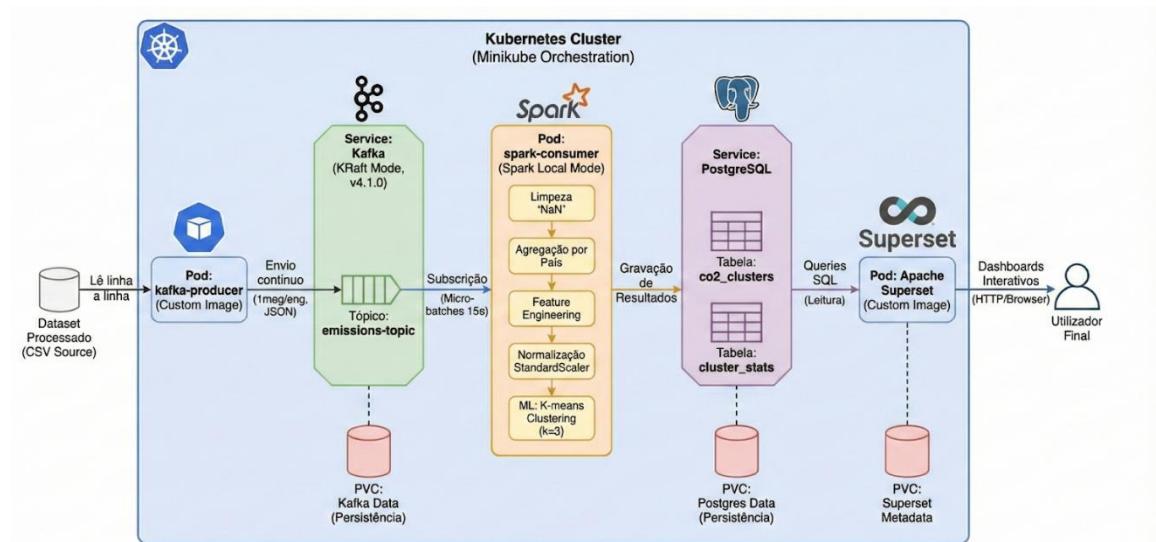


Figura 1. Pipeline de processamento de dados de emissões de CO2

3.1. Fluxo de dados

1º Ingestão (Kafka Producer): Com o dataset processado descrito na secção anterior, precisávamos de um serviço que lesse cada amostra e a enviasse. O kafka-producer, containerizado e executado como pod Kubernetes, lê o dataset linha a linha e, a cada segundo, envia uma mensagem JSON para o tópico *emissions-topic*, criando um cenário de streaming real, onde os dados chegam continuamente ao sistema.

2º Message Broker (Apache Kafka): Seguimos para o Apache Kafka que atua como intermediário (*message broker*), funcionando como um *buffer* tolerante a falhas entre o *producer* e o *consumer*. Decidimos utilizar o modo KRaft (*Kafka Raft Metadata mode*) ao invés de Apache Zookeeper, uma vez que estamos a utilizar a versão mais recente do kafka (4.1.0) que já não inclui o Zookeeper por padrão. O Kafka garante que as mensagens sejam persistidas em disco (mesmo que o *producer* falhe) e entregues de forma ordenada aos consumidores. O tópico *emissions-topic* funciona como uma fila distribuída que armazena temporariamente os registos que vai recebendo.

3º Processamento (Spark Consumer): Do lado do consumo, temos um job Apache Spark, designado por spark-consumer, que subscreeve o tópico Kafka. O Spark consome as mensagens em micro-batches, aplicando uma série de transformações. Optámos por executar Spark em modo local (dentro do próprio container do consumer) em vez de modo cluster distribuído, uma decisão justificada pelo tamanho moderado do dataset.

1. Limpeza de dados: remove strings "NaN" e converte-as em valores nulos apropriados;
2. Agregação por país: calcula médias das emissões, PIB e população para cada país ao longo do período histórico até ao momento (os dados são enviados em streaming);
3. *Feature engineering*: adiciona contexto temporal, como o primeiro e último ano com dados, e a média de emissões recentes (pós-2010);
4. Normalização: aplica *StandardScaler* às features numéricas para preparar o clustering;
5. Clustering K-means: agrupa os países em 3 clusters (k=3) com base nas suas características de emissões, desenvolvimento económico e demografia.

4º Armazenamento (PostgreSQL): Os resultados são então persistidos numa base de dados relacional, o PostgreSQL, onde criamos duas tabelas: *co2_clusters* (com os dados agregados e classificação de cada país) e *cluster_stats* (com estatísticas agregadas por cluster). Escolhemos o PostgreSQL por permitir *queries* eficientes e garantir a persistência dos resultados processados, o superset irá ler dados daí. Mesmo que o cluster Kubernetes seja parado (minikube stop), os dados permanecem no volume persistente.

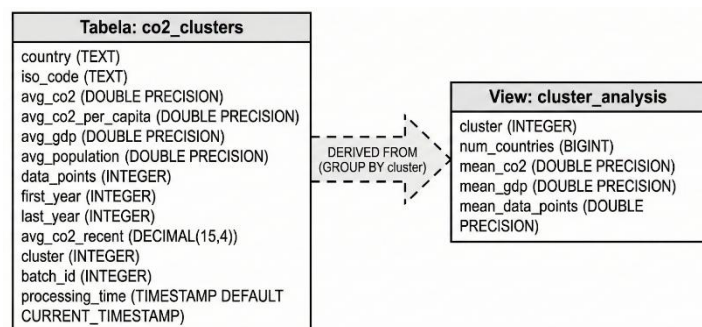


Figura 2 - Base de Dados

5º Visualização (Apache Superset): Finalmente, o Apache Superset conecta-se diretamente ao PostgreSQL e expõe os dados através de dashboards interativos, permitindo explorar visualmente os padrões identificados pelo algoritmo de clustering, comparar países dentro e entre clusters, entre outras análises pertinentes. Os metadados do Superset (dashboards, queries salvas, configurações de utilizadores) também são persistidos num PVC dedicado.

3.2. Containerização e orquestração

A containerização com Docker foi adotada para que cada serviço funcione de forma isolada, com as suas próprias dependências, ao facilitar a implementação e a escalabilidade. Criamos Dockerfiles personalizados para três dos nossos componentes, uma vez que as imagens oficiais não satisfaziam todos os requisitos: o kafka-producer (para incluir o nosso código e dataset), o spark-consumer (para adicionar os drivers de conexão ao Kafka e PostgreSQL) e o Superset (para incluir o driver psycopg2-binary para a ligação à BD).

Todo este ecossistema é orquestrado pelo Kubernetes (executado localmente via Minikube), que assume as seguintes responsabilidades:

1. Faz a gestão do lifecycle, ou seja, inicia, monitoriza e reinicia automaticamente pods que falhem.
2. Networking interno: Cria Services (ex: kafka:9092, postgres:5432) que permitem comunicação entre pods através de DNS interno.
3. Persistência de dados: Utiliza PVCs para garantir a durabilidade dos dados. Foram alocados volumes específicos para o Kafka (5Gi para mensagens), PostgreSQL (1Gi para dados processados) e Superset (2Gi para metadados), assegurando que a informação sobrevive mesmo se os pods forem reiniciados.
4. Deployment declarativo: 5 manifestos YAML que definem o estado desejado de toda a infraestrutura.

3.3. Tecnologias usadas

As tecnologias que usámos já foram mencionadas e estão representadas na figura seguinte.

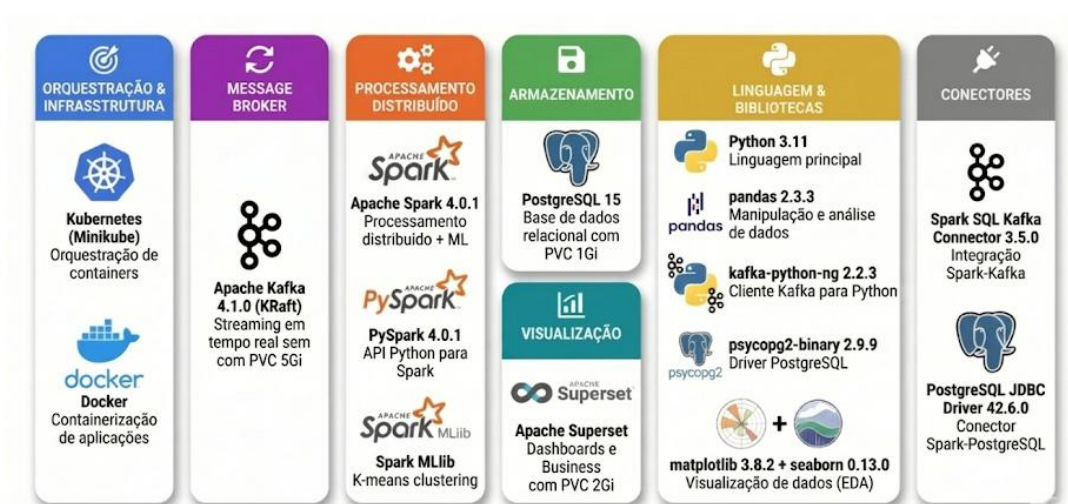


Figura 3 - Tecnologias envolvidas

4. Análise e Avaliação de Resultados e Padrões de Clustering

A aplicação do algoritmo K-means aos dados de emissões de CO₂, PIB e população permitiu a identificação de três clusters distintos, cada um representando um perfil característico de países. A escolha de $k=3$ foi baseada numa análise exploratória inicial e validada pelo Silhouette Score de 0.8402, uma métrica que varia de -1 a 1, onde valores próximos de 1 indicam clusters bem definidos e distintos. Este valor corresponde ao estado final da pipeline após o processamento completo dos 164 países, indicando uma separação robusta entre os clusters.

Esta segmentação é crucial para compreender as dinâmicas globais de emissão e desenvolvimento. Embora a análise detalhada dos países em cada cluster exija a exploração dos dados no Apache Superset, os perfis identificados para cada agrupamento são os seguintes:

Cluster 0: Baixos Emissores. Este cluster representa a grande maioria dos países, caracterizado por baixas emissões totais de CO₂ e pegadas per capita mínimas. Estas nações têm tipicamente populações menores e economias modestas, refletindo atividade industrial limitada e menor escala económica.

Cluster 1: Emissores Moderados. Apesar de conter relativamente poucos países, este cluster destaca-se por exibir as emissões per capita mais elevadas—substancialmente superiores aos outros dois grupos. Com populações e economias moderadas, estas nações demonstram estilos de vida e padrões industriais intensivos em carbono relativamente ao seu tamanho, sugerindo infraestruturas altamente desenvolvidas mas com elevadas emissões.

Cluster 2: Elevados Emissores. Este pequeno cluster de países outliers apresenta emissões totais mais elevadas, impulsionadas por populações e economias muito grandes. Curiosamente, as suas emissões per capita situam-se entre os outros clusters, indicando que a escala absoluta, e não a intensidade individual, impulsiona o seu impacto ambiental. A combinação de populações massivas com taxas per capita moderadas produz totais nacionais excecionalmente elevados.

A clusterização revela que a escala populacional e o desenvolvimento económico são os principais impulsionadores das emissões nacionais, enquanto a intensidade per capita varia independentemente dos totais absolutos—ênfatisando a distinção entre volume de emissões e intensidade de emissões.

4.1 Dashboard Interativo de Visualização

Para facilitar a exploração dos resultados da clusterização, foi desenvolvido um dashboard interativo no Apache Superset organizado em duas áreas principais.

A secção **Cluster Statistics** (Figura 4) apresenta a distribuição dos 164 países pelos três clusters identificados e o Silhouette Score de 0.8402 que valida a qualidade da segmentação. Uma tabela resumo exibe as métricas-chave de cada cluster (emissões totais, per capita, PIB e população), complementada por gráficos de barras comparativos.

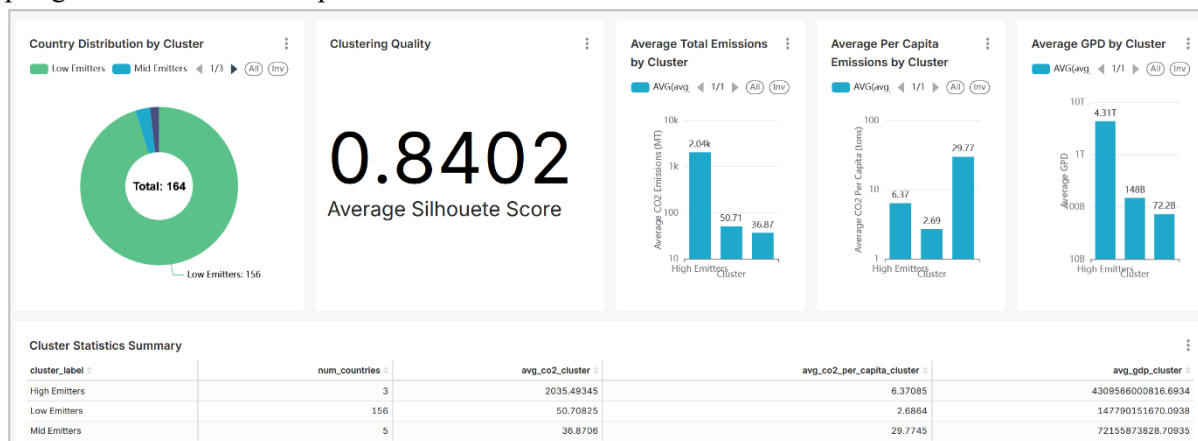


Figura 4 - Dashboard de Estatísticas dos Clusters

A **Country Analysis** (Figura 5) inclui múltiplas visualizações interativas. Um scatter plot relaciona PIB com emissões totais, e um bubble chart visualiza emissões totais versus per capita, onde o tamanho das bolhas representa a população de cada país, evidenciando a distinção entre escala absoluta e intensidade de emissões. Um radar chart normalizado permite comparações multidimensionais entre países selecionados. O choropleth map apresenta uma visualização geoespacial onde a intensidade da cor azul indica emissões médias, com bolhas vermelhas dimensionadas pela população, evidenciando que as emissões per capita americanas são superiores às chinesas apesar da maior população desta última. Uma tabela de análise multidimensional com funcionalidade de pesquisa completa esta secção.

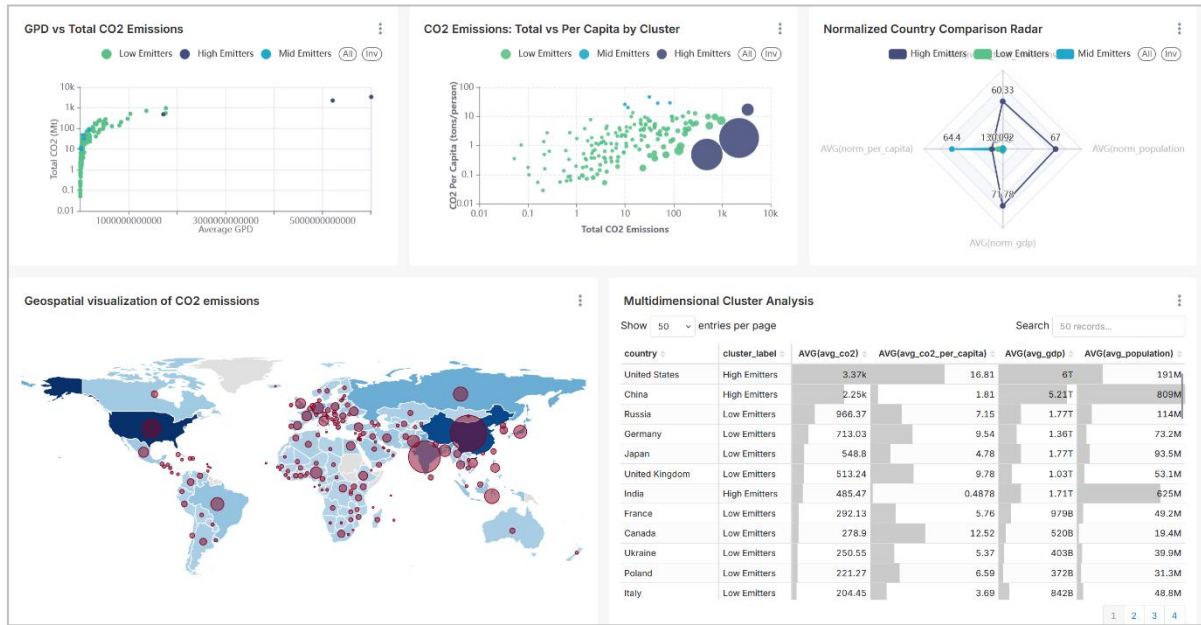


Figura 5 - Dashboard de Análise Comparativa de Países

5. Testes

Nesta secção, verificamos a persistência de dados (garantimos que não há perda de informação em caso de falha de pods ou reinício do cluster) e a integridade do streaming (para garantir a ausência de duplicados e a correta gestão de offsets).

Tabela 3. Testes de Resiliência e Integração

ID	Cenário de Teste	Objetivo e Verificação	Componentes/PVCs	Resultado
T1	Reinício do Pod Spark	Verificar se os dados processados persistem na BD após falha do consumidor.	<i>postgres-pvc</i>	Sucesso. Os dados foram mantidos.
T2	Reinício do Minikube	Simular falha total e verificar recuperação do estado	Todos os PVCs	Sucesso. Recuperação total
T3	Reinício do Producer	Garantir que o Kafka gere os offsets e não cria registos duplicados no reinício.	<i>kafka-data</i>	Sucesso. Zero duplicados.

Tabela 3. Testes de Resiliência e Integração

T4	Conexão Superset	Validar a integração visual e leitura da base de dados PostgreSQL.	<i>superset-pvc</i>	Sucesso. A conexão manteve-se ativa.
-----------	------------------	--	---------------------	--

Como observado nos testes T1 e T2, a utilização de Persistent Volume Claims (PVCs) provou ser eficaz. Ao reiniciarmos o pod spark-consumer e, posteriormente, todo o cluster Minikube, verificamos através de queries SQL (SELECT COUNT(*)) que o número de registos na tabela co2_clusters se mantiveram inalterados em 8776 registos, confirmando a resiliência do armazenamento PostgreSQL.

No teste T3, forçamos o kafka-producer a reiniciar durante o envio de mensagens. Confirmamos que o broker Kafka geriu corretamente os offsets e retomou o processamento exatamente onde parou, sem introduzir registos duplicados na base de dados final.

Por fim, validamos a integração final da pipeline. O Apache Superset conectou-se com sucesso ao PostgreSQL.

6. Conclusão

Este projeto demonstra a viabilidade de uma pipeline de dados em tempo real para análise de emissões globais de CO₂, integrando tecnologias de streaming (Kafka), processamento distribuído (Spark) e visualização interativa (Superset), orquestradas através de Kubernetes. A clusterização identificou com sucesso três perfis distintos de países, validados por um silhouette score robusto de 0.8402. Algumas limitações foram identificadas: a execução do Spark em modo local limita a escalabilidade para datasets maiores, o valor $k=3$ foi fixo sem exploração de outras configurações, e o streaming simula dados históricos em vez de capturar emissões verdadeiramente em tempo real.

Como trabalho futuro, seria interessante explorar diferentes valores de k através de métodos como o Elbow Method, incorporar mais features, implementar modelos de previsão de emissões futuras, e testar a pipeline com dados de maior volume para avaliar o desempenho em escala real.

7. Bibliografia

- [1] Ritchie, H., Rosado, P., & Roser, M. (2020). CO₂ and Greenhouse Gas Emissions dataset. Our World in Data. <https://github.com/owid/co2-data>.
- [2] As representações visuais e diagramas conceptuais presentes neste relatório (Figuras 1 e 3) foram elaboradas com o auxílio de ferramentas de Inteligência Artificial (Google Gemini).
- [3] Apache Software Foundation. (2024). Apache Kafka Documentation. <https://kafka.apache.org/documentation/>.
- [4] Apache Software Foundation. (2024). Spark Structured Streaming Programming Guide. <https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html>.
- [5] Kubernetes Documentation. (2024). Kubernetes Concepts. <https://kubernetes.io/docs/concepts/>.
- [6] Apache Software Foundation. (2024). Apache Superset Documentation. <https://superset.apache.org/docs/intro>.